

DRS — Dynamic Resource Scheduler

secret : d825a548-f24a-4071-b5a6-c6f12b33ec9e / Token ID : root@pam!drs

Application Laravel pour déployer des VMs et conteneurs LXC sur un cluster Proxmox avec sélection automatique du nœud optimal.

Architecture

Laravel App

ProxmoxService	← appels API REST Proxmox
NodeSelectorService	← logique de choix du meilleur nœud
VmController	← endpoints HTTP (web + API)
CreateProxmoxVm	← job asynchrone de création
Blade UI	← formulaire de création + tableau de bord

Dashboard :

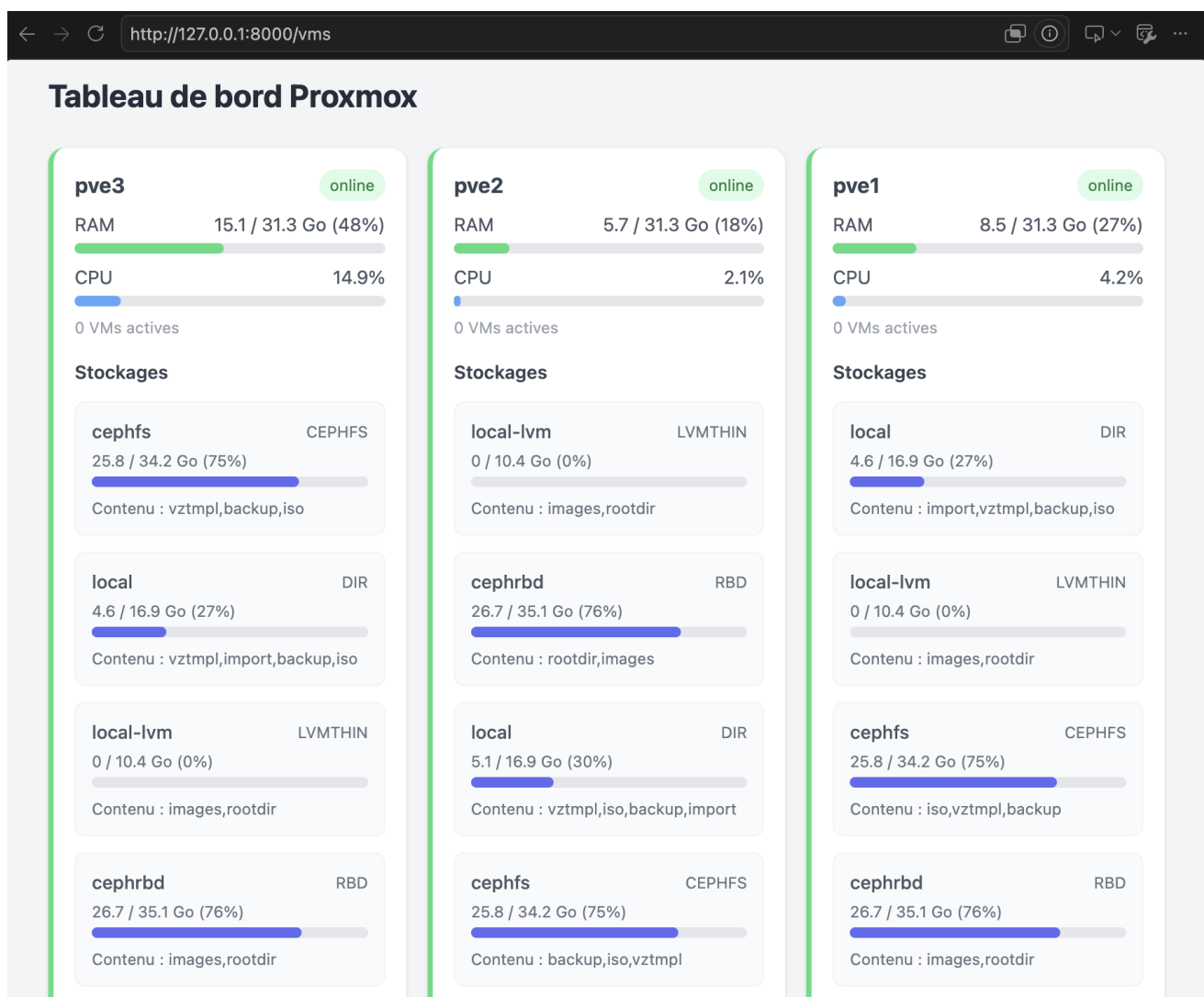
Tableau de bord Proxmox



Jobs récents

#	Nom	Type	Nœud	VMID	Statut	Progression	Actions
5	web-server02	VM	pve2	111	done	100% Machine "web-server02" déployée sur pve2.	—
4	web-server01	VM	pve2	111	error	50% Erreur : {"storage": "storage 'cephfs' does not support vm images"}	Relancer Modifier
3	TestDRS	VM	pve2	111	error	50% Erreur : storage 'local-zfs' does not exist	Relancer Modifier
2	DRSVMS	VM	pve3	111	error	50% Erreur : Permission check failed	Relancer Modifier
1	TestVms001	VM	pve3	111	error	50% Erreur : Permission check failed	Relancer Modifier

add storage section :



create VMs/CTs

← → ↻ http://127.0.0.1:8000/vms/create

☒ VM (QEMU) ☐ Conteneur (LXC)

Nom

ex: web-server-01

RAM (Mo)

2 Go

vCPU

2 cœurs

Disque (Go)

20

Stockage

local-lvm

Bridge réseau

vmbr0

Template VM (clone)

— Aucun (disque vierge) —

Méthode de sélection du nœud

☒ RAM libre ☐ CPU libre ☐ Score combiné (RAM 60% + CPU 40%)

Nœud sélectionné automatiquement : pve2 (méthode : memory)

Prochain VMID disponible : 113

Créer et déployer automatiquement

Prérequis

- PHP 8.2+
- Composer
- Extension SQLite (ou MySQL/PostgreSQL)

Installation

```
composer install
cp .env.example .env
php artisan key:generate
```

Configurer Proxmox dans `.env` :

```
PROXMOX_HOST=votre-serveur.proxmox
PROXMOX_PORT=8006
PROXMOX_USER=root@pam
PROXMOX_TOKEN_ID=votre-token
```

```
PROXMOX_TOKEN_SECRET=votre-secret  
PROXMOX_VERIFY_SSL=false
```

Puis :

```
php artisan migrate  
php artisan db:seed
```

Démarrage

Terminal 1 — serveur web :

```
php artisan serve
```

Terminal 2 — worker de queue (création asynchrone) :

```
php artisan queue:work
```

Interface web : <http://localhost:8000/vms>

API (Sanctum)

Compte par défaut après seed : [admin@drs.local](#) / [password](#)

```
# Login  
curl -X POST http://localhost:8000/api/login \  
  -H "Content-Type: application/json" \  
  -d '{"email":"admin@drs.local","password":"password"}'  
  
# Créer une VM (token Bearer)  
curl -X POST http://localhost:8000/api/vms \  
  -H "Authorization: Bearer TOKEN" \  
  -H "Content-Type: application/json" \  
  -d '{  
    "name": "web-01",  
    "type": "vm",  
    "memory": 2048,  
    "cores": 2,  
    "disk_size": 20,  
    "storage": "local-zfs",  
    "bridge": "vmbro",  
    "method": "score"  
  }'
```

```
# Suivi du job
curl http://localhost:8000/api/jobs/1 \
-H "Authorization: Bearer TOKEN"
```

Endpoints

Méthode	Route	Description
GET	/vms	Tableau de bord
GET	/vms/create	Formulaire de création
POST	/vms	Lancer un déploiement
GET	/api/nodes	Statut des nœuds (public web)
GET	/api/best-node	Meilleur nœud selon méthode
GET	/api/templates	Templates VM/CT disponibles
POST	/api/login	Authentification Sanctum
POST	/api/vms	Création via API (auth)
GET	/api/jobs/{id}	Statut d'un job

Méthodes de placement

- **memory** — nœud avec le plus de RAM libre
- **cpu** — nœud avec la charge CPU la plus faible
- **score** — score combiné RAM (60%) + CPU (40%)